Qiskit Fall Fest 2026

University of Ottawa · October 3, 2026

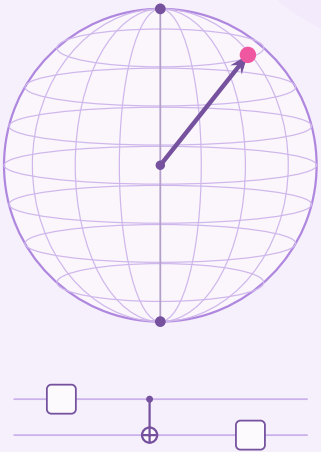
An Introduction to

Quantum Machine Learning

Sohrab Ganjian

Quantum Research Scientist

Natural Resources Canada



Overview

1. Machine learning foundations
2. Quantum computing foundations
3. Classification with quantum kernels
4. Training quantum classifiers
5. Broader topics in quantum computing
6. Guiding questions and concluding remarks

01 / 06

Machine learning foundations

What do we mean by learning?

- **Learning** is the process of acquiring knowledge, skills, and behaviours through experience, study, or instruction.

What do we mean by learning?

- **Learning** is the process of acquiring knowledge, skills, and behaviours through experience, study, or instruction.
- **Machine learning (ML)** involves algorithms that learn from data to make predictions, discover patterns, or select actions based on feedback.

What do we mean by learning?

- **Learning** is the process of acquiring knowledge, skills, and behaviours through experience, study, or instruction.
- **Machine learning (ML)** involves algorithms that learn from data to make predictions, discover patterns, or select actions based on feedback.
- **Artificial intelligence (AI)** involves creating artificial systems capable of tasks associated with intelligence, such as perception, reasoning, and decision-making.

What do we mean by learning?

- **Learning** is the process of acquiring knowledge, skills, and behaviours through experience, study, or instruction.
- **Machine learning (ML)** involves algorithms that learn from data to make predictions, discover patterns, or select actions based on feedback.
- **Artificial intelligence (AI)** involves creating artificial systems capable of tasks associated with intelligence, such as perception, reasoning, and decision-making.
- ML and AI challenge our understanding of intelligence, agency, and consciousness. These topics are beyond the scope of this talk.

Learning to predict

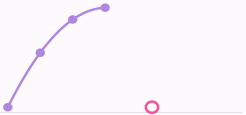
01 **Initial estimate**

02 **Observed outcome**

03 **Revised estimate**



Human



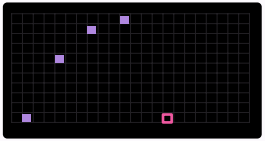




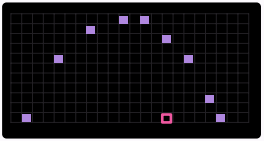
Update the model



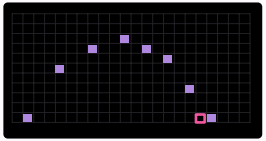
Computer



Sampled positions



Error: 5 cells



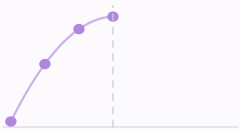
New throw
Error: 1 cell

Update the model

Observed Predicted

Building intuition for models

Observed positions



What we have seen so far

Model

A simple flight model

Launch height

Horizontal speed

Initial upward speed

Different settings, different paths

Estimated landing

Compare where the ball lands

Predicted

Actual

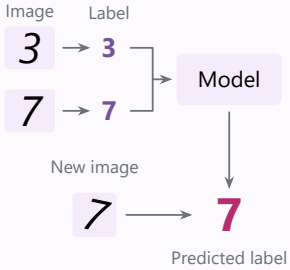
Prediction error

Tune the parameters

$$\text{predicted path} = \text{Model}\left(\underbrace{\text{launch height},\ \text{horizontal speed},\ \text{initial upward speed}}_{\text{fitted to the observations}}\right)$$

Types of Machine Learning Algorithms

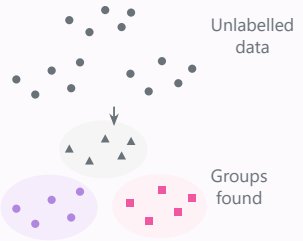
Supervised learning



Learns from labelled examples to make predictions about new cases.

Tumour detection
Speech recognition

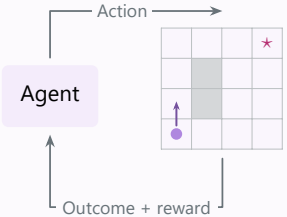
Unsupervised learning



Finds patterns or structure in unlabelled data.

Finding crime hotspots
Grouping songs by sound

Reinforcement learning



Learns how to act through interaction and rewards.

Playing Go
Teaching a robot to walk

02 / 06

Quantum computing foundations

Quantum computing

> *"Nature isn't classical, dammit, and if you want to make a simulation of nature, you'd better make it quantum mechanical."*
>
> Richard Feynman

Quantum computers use superposition, interference, and entanglement to solve certain problems more efficiently than classical approaches.

Superposition



Interference



Entanglement



Quantum computing

> *"Nature isn't classical, dammit, and if you want to make a simulation of nature, you'd better make it quantum mechanical."*
>
> Richard Feynman

Quantum computers use superposition, interference, and entanglement to solve certain problems more efficiently than classical approaches.

Superposition



Interference



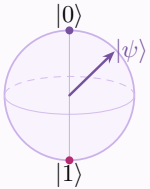
Entanglement



> *"Quantum computers won't solve hard problems instantly by just trying all solutions in parallel."*
>
> Scott Aaronson

Qubits and quantum gates

One qubit



$|\psi\rangle = a|0\rangle + b|1\rangle$

a, b are amplitudes.

Outcomes 0 and 1 have probabilities $|a|^2$ and $|b|^2$.

$|a|^2 + |b|^2 = 1$

Multiple qubits

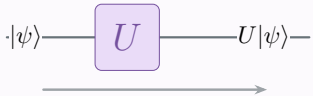
Two qubits, four basis states

$\|00\rangle$ a_{00}	$\|01\rangle$ a_{01}
$\|10\rangle$ a_{10}	$\|11\rangle$ a_{11}

One joint state, with an amplitude for each bit string.

Each added qubit doubles the number of basis states.

Quantum gates



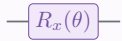
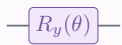
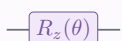
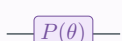
$|\psi'\rangle = U|\psi\rangle$

A unitary gate preserves the norm of the quantum state.

n qubits can be in a superposition of 2^n bit strings,
but measuring them gives only one n-bit string out of 2^n possibilities.

Common gates

Gate	Matrix	Circuit
I	$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$	
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	
Y	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$	
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$	
H	$\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	

Gate	Matrix	Circuit
$R_x(\theta)$	$\begin{pmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}$	
$R_y(\theta)$	$\begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}$	
$R_z(\theta)$	$\begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$	
$P(\theta)$	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$	
CNOT	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$	

Classical and quantum data and algorithms

Type of Algorithm

Type of Data	Classical	Quantum
Classical	**CC** Example Recognizing handwritten digits with a classical model.	**CQ** Example Recognizing handwritten digits with a quantum model.
Quantum	**QC** Example Helping correct quantum computing errors with a classical model.	**QQ** Example Identifying phases of a quantum material from its quantum states.

03 / 06

Classification with quantum kernels

Preparing data for encoding

- **Feature selection.** Feature selection is the process of choosing informative features relevant to a task. Current quantum computers have limited qubit counts and circuit depths, so reducing the number of features can help make experiments feasible on available hardware.

Preparing data for encoding

- **Feature selection.** Feature selection is the process of choosing informative features relevant to a task. Current quantum computers have limited qubit counts and circuit depths, so reducing the number of features can help make experiments feasible on available hardware.
- **Quantum state normalization.** Quantum state vectors have length 1, that is $\|\psi\| = 1$. Input data must be normalized when required by the encoding to satisfy this condition.

Preparing data for encoding

- **Feature selection.** Feature selection is the process of choosing informative features relevant to a task. Current quantum computers have limited qubit counts and circuit depths, so reducing the number of features can help make experiments feasible on available hardware.
- **Quantum state normalization.** Quantum state vectors have length 1, that is $\|\psi\| = 1$. Input data must be normalized when required by the encoding to satisfy this condition.
- **Data scaling and transformations.** A wide feature range can make task-relevant differences difficult to distinguish after encoding. Suitable transformations can give those differences greater separation.



Preparing data for encoding

- **Feature selection.** Feature selection is the process of choosing informative features relevant to a task. Current quantum computers have limited qubit counts and circuit depths, so reducing the number of features can help make experiments feasible on available hardware.
- **Quantum state normalization.** Quantum state vectors have length 1, that is $\|\psi\| = 1$. Input data must be normalized when required by the encoding to satisfy this condition.
- **Data scaling and transformations.** A wide feature range can make task-relevant differences difficult to distinguish after encoding. Suitable transformations can give those differences greater separation.



- **Rotation angle ranges.** Single-qubit rotation gates are 2π-periodic up to global phase. A full cycle is represented by any interval of width 2π, such as $[0, 2\pi)$ or $[-\pi, \pi)$. Smaller ranges, such as $[0, \pi]$, are also used for data encoding.

Preparing data for encoding

- **Feature selection.** Feature selection is the process of choosing informative features relevant to a task. Current quantum computers have limited qubit counts and circuit depths, so reducing the number of features can help make experiments feasible on available hardware.
- **Quantum state normalization.** Quantum state vectors have length 1, that is $\|\psi\| = 1$. Input data must be normalized when required by the encoding to satisfy this condition.
- **Data scaling and transformations.** A wide feature range can make task-relevant differences difficult to distinguish after encoding. Suitable transformations can give those differences greater separation.



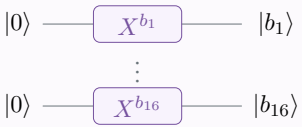
- **Rotation angle ranges.** Single-qubit rotation gates are 2π-periodic up to global phase. A full cycle is represented by any interval of width 2π, such as $[0, 2\pi)$ or $[-\pi, \pi)$. Smaller ranges, such as $[0, \pi]$, are also used for data encoding.
- **Choose a suitable encoding.** Examples include **basis, angle, amplitude**, phase, and dense angle encoding. Methods differ in qubit requirements and circuit complexity. The choice depends on resource constraints and the task objective.

Different ways to encode data

Consider two integer-valued features, $\mathbf{x} = (207, 8)$, each ranging from 0 to 255.

Basis encoding

Values become bits



$b_j = 0$: I $\quad$ $b_j = 1$: X

$|11001111\,00001000\rangle$

16 qubits

n bits per feature $\rightarrow$ n qubits

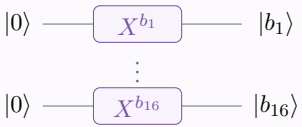
Here $n = 8$ $\quad 2^8 = 256$ values

Different ways to encode data

Consider two integer-valued features, $\mathbf{x} = (207, 8)$, each ranging from 0 to 255.

Basis encoding

Values become bits



$b_j = 0$: I $\quad$ $b_j = 1$: X

$|11001111\,00001000\rangle$

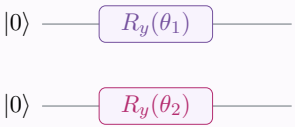
16 qubits

n bits per feature $\rightarrow$ n qubits

Here $n = 8$ $\quad 2^8 = 256$ values

Angle encoding

Values set rotation angles



$\theta_i = \pi x_i / 255$

$R_y(.812\pi)|0\rangle \otimes R_y(.031\pi)|0\rangle$

2 qubits

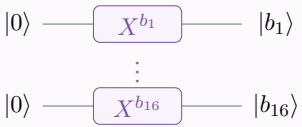
One qubit per feature

Different ways to encode data

Consider two integer-valued features, $\mathbf{x} = (207, 8)$, each ranging from 0 to 255.

Basis encoding

Values become bits



$b_j = 0$: I $\quad$ $b_j = 1$: X

$|11001111\,00001000\rangle$

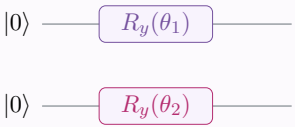
16 qubits

n bits per feature $\rightarrow$ n qubits

Here $n = 8$ $\quad 2^8 = 256$ values

Angle encoding

Values set rotation angles



$\theta_i = \pi x_i / 255$

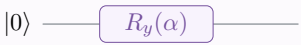
$R_y(.812\pi)|0\rangle \otimes R_y(.031\pi)|0\rangle$

2 qubits

One qubit per feature

Amplitude encoding

Values become amplitudes



$$\alpha = 2\arccos\left(\frac{207}{\sqrt{207^2 + 8^2}}\right)$$

$$R_y(\alpha)|0\rangle = \frac{207|0\rangle + 8|1\rangle}{\sqrt{207^2 + 8^2}}$$

1 qubit

N features $\rightarrow$ $\lceil \log_2 N \rceil$ qubits

Normalization removes overall scale

$\mathbf{x}$ and $2\mathbf{x}$ give the same state

Loading data as a quantum state

Image



One dog photograph

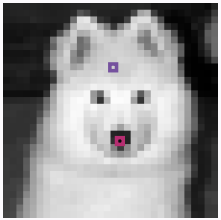
Loading data as a quantum state

Image



One dog photograph

Pixel values



32×32 grayscale pixels

Selected pixel values

$$\mathbf{x} = (207, 8)$$

0 = black 255 = white

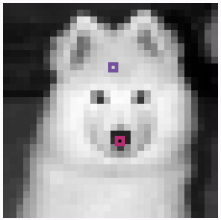
Loading data as a quantum state

Image



One dog photograph

Pixel values



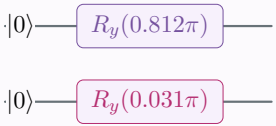
32×32 grayscale pixels

Selected pixel values

$$\mathbf{x} = (207,\ 8)$$

0 = black 255 = white

Angle encoding



$$\theta_i = \pi \frac{x_i}{255}$$

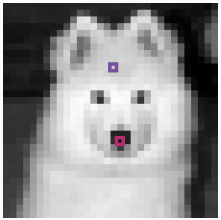
Loading data as a quantum state

Image



One dog photograph

Pixel values



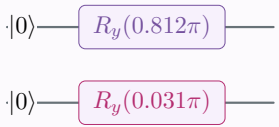
32×32 grayscale pixels

Selected pixel values

$\mathbf{x} = (207,\ 8)$

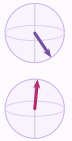
0 = black 255 = white

Angle encoding



$$\theta_i = \pi \frac{x_i}{255}$$

Quantum state



$$|\psi(\mathbf{x})\rangle$$

Encoding turns classical data into a quantum state.

Comparing quantum states

How similar are two quantum states?

$$k(\mathbf{x}, \mathbf{z}) = \left| \langle \psi(\mathbf{z}) \mid \psi(\mathbf{x}) \rangle \right|^2$$

0 = no overlap 1 = physically indistinguishable states

How can a circuit measure this overlap?

$$|00\rangle \xrightarrow{\quad U(\mathbf{z}) \quad} |\psi(\mathbf{z})\rangle \xrightarrow{\quad U^\dagger(\mathbf{z}) \quad} |00\rangle$$

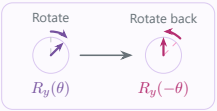
$$|00\rangle \xrightarrow{\quad U(\mathbf{x}) \quad} |\psi(\mathbf{x})\rangle \xrightarrow{\quad U^\dagger(\mathbf{z}) \quad} U^\dagger(\mathbf{z})|\psi(\mathbf{x})\rangle$$

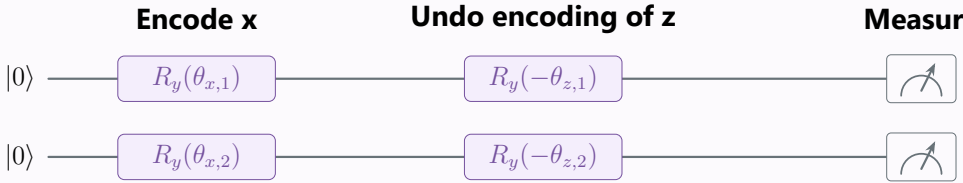
The same unitary changes both states without changing their overlap.

$$\underbrace{\langle \psi(\mathbf{z}) \mid \psi(\mathbf{x}) \rangle}_{\text{overlap of the encoded states}} = \underbrace{\langle 00 | U^\dagger(\mathbf{z}) | \psi(\mathbf{x}) \rangle}_{\text{amplitude of returning to } 00}$$

Comparing quantum states

How similar are two quantum states?





$$k(\mathbf{x}, \mathbf{z}) = \left|\langle \psi(\mathbf{z}) \mid \psi(\mathbf{x}) \rangle\right|^2 = P(00) \approx \frac{\text{\# observed } 00 \text{ outcomes}}{\text{\# total runs}}$$

Repeat the circuit and count the 00 outcomes.

Comparing encoded data

Same pixel positions. Same angle encoding.

Reference dog



$\mathbf{x} = (207,\ 8)$

$|\psi(\mathbf{x})\rangle$



Another dog

$\mathbf{z}_{\text{dog}} = (201,\ 55)$

$|\psi(\mathbf{z}_{\text{dog}})\rangle$

Similarity

0.917

0 = no overlap 1 = physically indistinguishable states

Similarity $k(\mathbf{x}, \mathbf{z}) = \left|\langle\psi(\mathbf{x}) \mid \psi(\mathbf{z})\rangle\right|^2$

Comparing encoded data

Same pixel positions. Same angle encoding.

Reference dog



$\mathbf{x} = (207,\ 8)$

$|\psi(\mathbf{x})\rangle$



Another dog

$\mathbf{z}_{\text{dog}} = (201,\ 55)$

$|\psi(\mathbf{z}_{\text{dog}})\rangle$

Similarity

0.917



A cat

$\mathbf{z}_{\text{cat}} = (154,\ 162)$

$|\psi(\mathbf{z}_{\text{cat}})\rangle$

Similarity

0.305

0 = no overlap 1 = physically indistinguishable states

Similarity $k(\mathbf{x}, \mathbf{z}) = \left|\langle\psi(\mathbf{x}) \mid \psi(\mathbf{z})\rangle\right|^2$

The quantum kernel matrix

What if we compare every example with every other example?








The quantum kernel matrix

$$K_{ij} = \left|\langle\psi(\mathbf{x}_i) \mid \psi(\mathbf{x}_j)\rangle\right|^2$$

i: row example j: column example

The quantum kernel matrix

What if we compare every example with every other example?



The quantum kernel matrix

$$K_{ij} = \left|\langle\psi(\mathbf{x}_i) \mid \psi(\mathbf{x}_j)\rangle\right|^2$$

i: row example j: column example

The quantum kernel matrix

What if we compare every example with every other example?









1.000	0.917	0.305
0.917	1.000	0.574
0.305	0.574	1.000

Darker = higher similarity $0 \leq K_{ij} \leq 1$

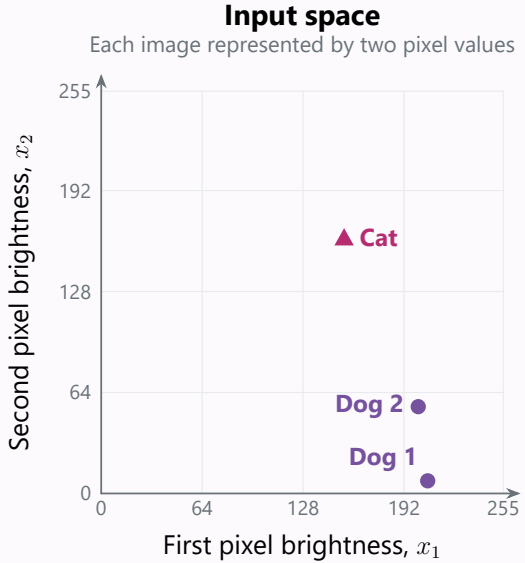
The quantum kernel matrix

$$K_{ij} = \left| \langle \psi(\mathbf{x}_i) \mid \psi(\mathbf{x}_j) \rangle \right|^2$$

i: row example j: column example

The kernel matrix collects pairwise similarities between encoded examples.

Images in input space





Dog 1
$(207, 8)$



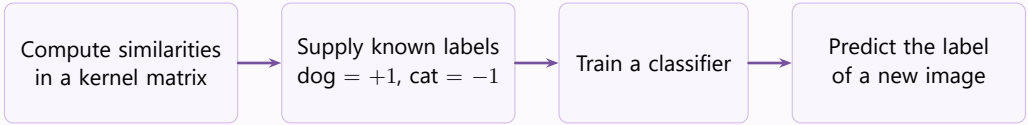
Dog 2
$(201, 55)$



Cat
$(154, 162)$

From similarities to classification

We can compare images. How do we turn those comparisons into a label?



One method: a support vector machine (SVM)

An SVM is a classifier that learns a decision boundary, balancing a wide margin against missed margin targets.

$$f(\mathbf{x}) = \sum_{i=1}^{n} a_i \, k(\mathbf{x}_i, \mathbf{x}) + b$$

SVM decision function (outputs a score)

k: fixed similarities $\quad$ a_i: learned weights $\quad$ b: learned offset

The kernel matrix supplies similarities; labels and training determine the classifier.

Training the SVM

Training adjusts the weights $\mathbf{a} = (a_1, \ldots, a_n)$ and offset b. The kernel matrix K stays fixed.

$$\min_{\mathbf{a},b} \ \frac{1}{2}\mathbf{a}^{\mathsf{T}} K \mathbf{a} + C \sum_i \max(0, 1 - y_i f(\mathbf{x}_i))$$

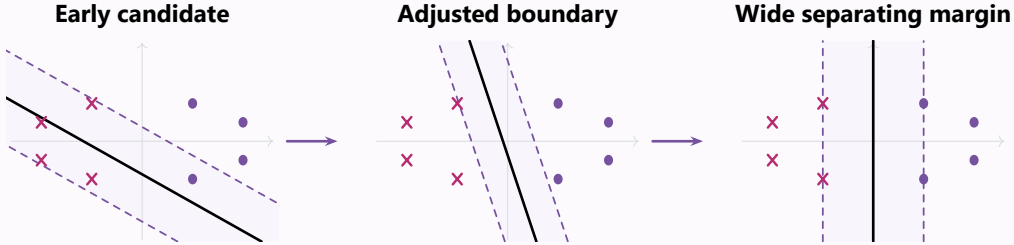
Encourages a wider margin.

$y_i = \pm 1$: the known label of example i.

Penalizes points inside the margin or on the wrong side.

Larger $C > 0$ makes those penalties stronger.

Classical methods optimize the weights and offset. Examples include quadratic programming software and sequential minimal optimization (SMO).



Solid: $f = 0$ Dashed: $f = \pm 1$ Shaded: margin band

From kernel values to decision scores



Dog 1

$\mathbf{x} = (207, 8), \quad y = +1$



Dog 2

$\mathbf{x} = (201, 55), \quad y = +1$



Cat

$\mathbf{x} = (154, 162), \quad y = -1$

SVM decision score for Dog 1

$$f(\mathbf{x}_{\text{dog 1}}) = a_1\,(1.000) + a_2\,(0.917) + a_3\,(0.305) + b$$

Training adjusts a_1, a_2, a_3, b. The kernel values are pre-computed.

From kernel values to decision scores



Dog 1

$\mathbf{x} = (207, 8), \quad y = +1$



Dog 2

$\mathbf{x} = (201, 55), \quad y = +1$



Cat

$\mathbf{x} = (154, 162), \quad y = -1$

$$f(\mathbf{x}) = a_1 k(\mathbf{x}_{\text{dog 1}}, \mathbf{x}) + a_2 k(\mathbf{x}_{\text{dog 2}}, \mathbf{x}) + a_3 k(\mathbf{x}_{\text{cat}}, \mathbf{x}) + b$$

During training: evaluate f at Dog 1, Dog 2 and Cat using their fixed pixel pairs.

Evaluate
Score all three animals

→

Check
Check score targets

→

Update
Adjust weights and offset

Repeat to minimize the SVM objective

$$a_1 = 0, \quad a_2 \approx 2.347, \quad a_3 \approx -2.347, \quad b \approx 0$$

Decision scores and margins

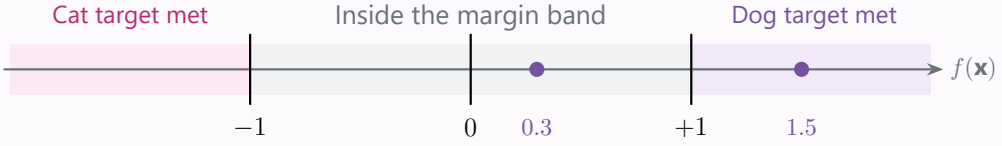
An SVM aims to place examples on the side of the decision boundary corresponding to their class, with a margin that provides a buffer between the boundary and the training examples.



Cat target: $f(\mathbf{x}) \leq -1$



Dog target: $f(\mathbf{x}) \geq 1$



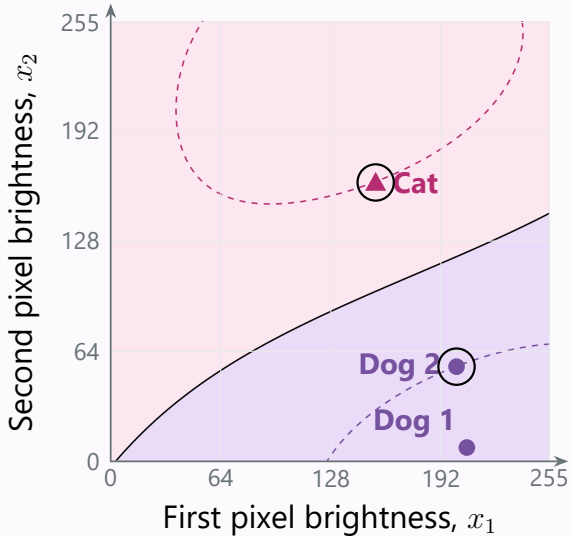
Example: a dog scores 0.3

Predicted dog, but inside the margin.
Penalty: $\max(0, 1 - 0.3) = 0.7$.

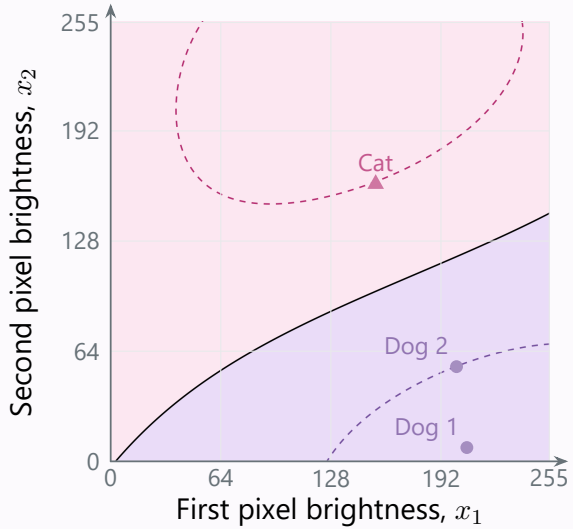
Example: a dog scores 1.5

Predicted dog and meets the target.
Penalty: $\max(0, 1 - 1.5) = 0$.

Visualizing the trained classifier



Classifying new images





New image A

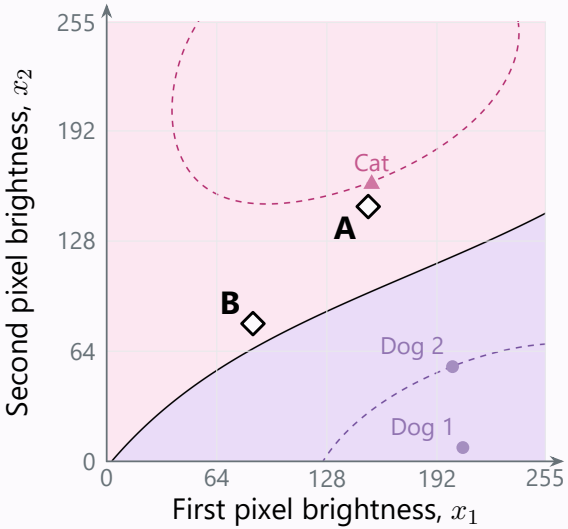
Label to predict: ?



New image B

Label to predict: ?

Classifying new images





New image A

$\mathbf{x}_A = (152, 148)$

$f(\mathbf{x}_A) = -0.818 \Rightarrow$ cat

Actual: dog. Incorrect.



New image B

$\mathbf{x}_B = (85, 80)$

$f(\mathbf{x}_B) = -0.184 \Rightarrow$ cat

Actual: cat. Correct.

◇ New examples

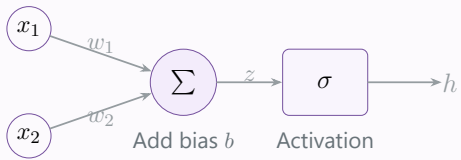
04 / 06

Training quantum classifiers

Classical neural networks

Another way to classify: learn a sequence of transformations of the input.

One neuron



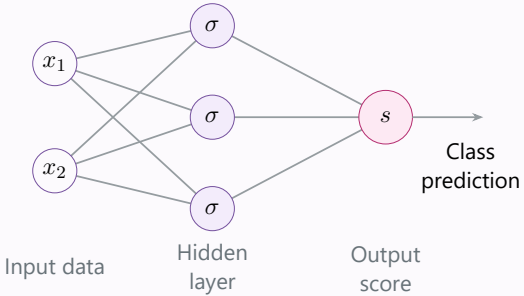
$$z = w_1 x_1 + w_2 x_2 + b, \qquad h = \sigma(z)$$

Example activation: ReLU

$$\sigma(z) = \max(0, z)$$



A small neural network



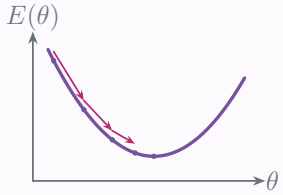
Training adjusts weights and biases to reduce prediction errors on labelled examples.

Why train a quantum circuit?

An early example: the variational quantum eigensolver (VQE)

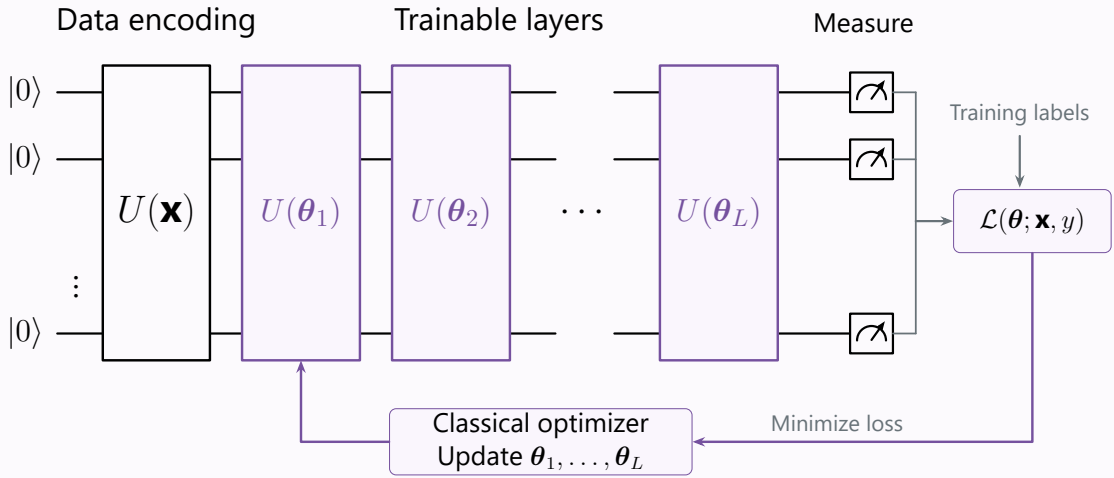
- A Hamiltonian H is an operator representing a system's total energy.
- The ground state of H is a quantum state with the lowest possible energy.
  Molecular ground-state energies provide information about molecular structure and stability, with applications in materials science and drug discovery.
- **VQE** prepares a parameterized quantum state $|\psi(\boldsymbol{\theta})\rangle$ and estimates its energy.
- An optimizer adjusts $\boldsymbol{\theta}$ to lower the energy estimate.

$$E_0 \approx \min_{\boldsymbol{\theta}\in\Theta} \langle\psi(\boldsymbol{\theta})|H|\psi(\boldsymbol{\theta})\rangle$$



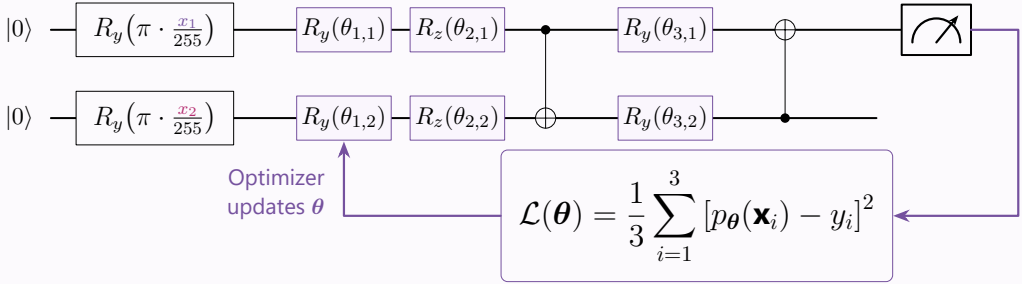
- VQE showed how optimizing a quantum circuit's parameters can lower its energy estimate, helping motivate trainable quantum circuits for other objectives, including machine learning.

Training a quantum circuit



Variational Quantum Classifier (VQC)

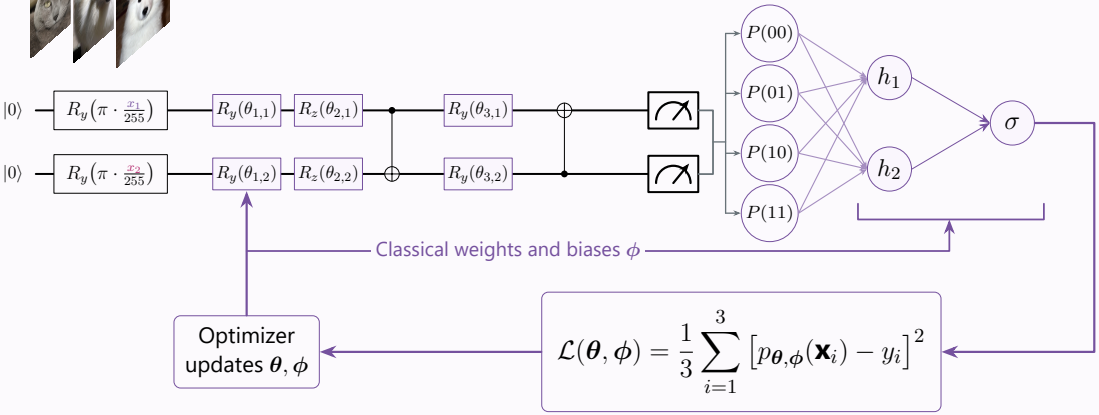




Predict dog if $p_{\boldsymbol{\theta}}(\mathbf{x}) > 0.5$, otherwise cat.

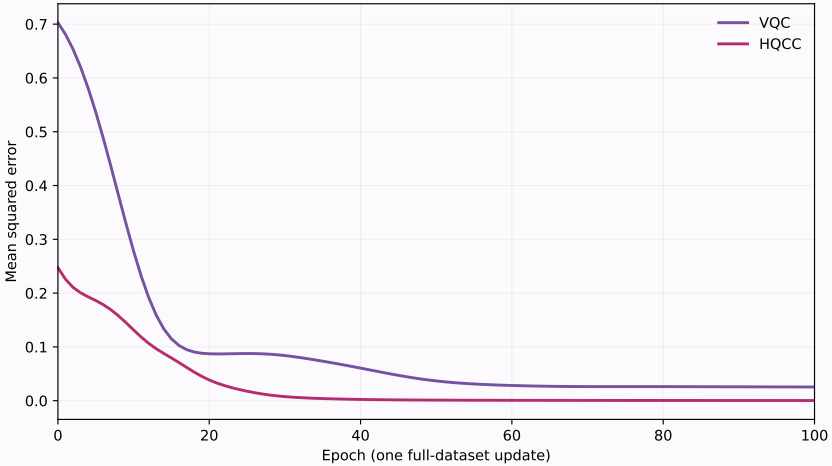
Hybrid Quantum–Classical Classifier (HQCC)



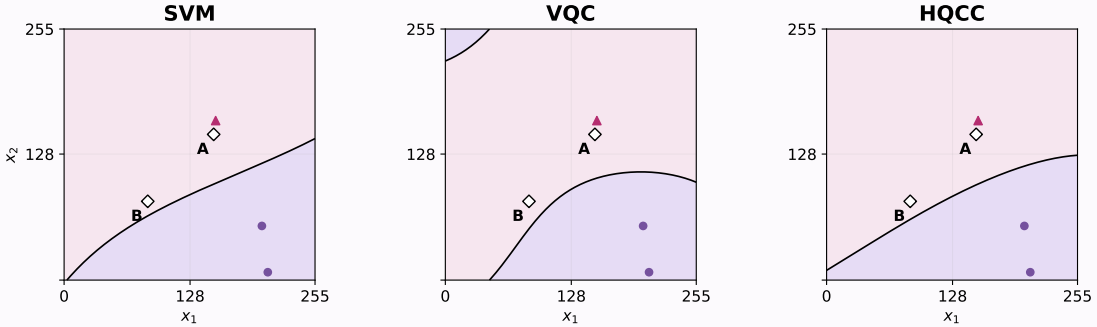


Predict dog if $p_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathbf{x}) > 0.5$, otherwise cat.

Training loss curves



Comparing the trained classifiers



05 / 06

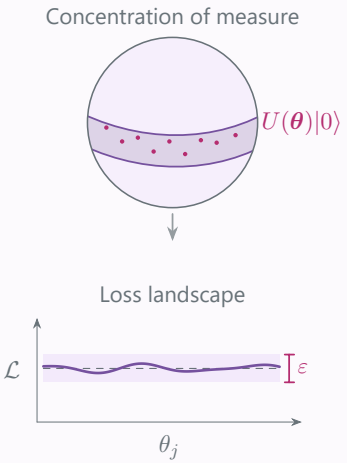
Broader topics in quantum computing

Barren plateaus in quantum computing

- In a **barren plateau**, gradients concentrate near zero, with variance shrinking exponentially as qubit count grows.
- For sufficiently random variational circuit families, increasing depth can lead to a barren plateau, with gradient variance shrinking exponentially as qubit count grows.
- Careful choices of circuit structure, loss function or initialization can avoid barren plateaus in some settings.
- Suppose we design a quantum model that provably avoids barren plateaus.

  Does provable absence of barren plateaus imply classical simulability?

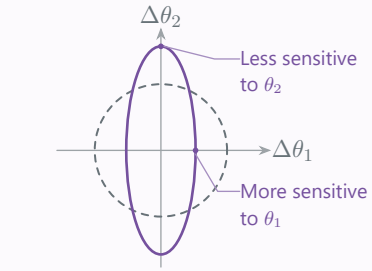
  Many studied models admit efficient classical evaluation of the loss. However, absence of barren plateaus alone may not guarantee classical simulability.



McClean et al. (2018) Cerezo et al. (2025)

Quantum natural gradient

- An optimizer updates circuit parameters to reduce the loss. Its update rule determines the direction and size of each training step.
- Equal changes in circuit parameters can produce very different changes in the quantum state. Quantum-state distance measures these changes through the overlap between states.
- Gradient descent (GD) chooses the steepest local decrease in loss using ordinary distance in parameter space. Quantum natural gradient (QNG) uses quantum-state distance instead.



GD : $\Delta\boldsymbol{\theta} = -\eta\nabla\mathcal{L}$

QNG : $\Delta\boldsymbol{\theta} = -\eta g^{+}\nabla\mathcal{L}$

$\Delta\boldsymbol{\theta}$: parameter update η: step size
$\nabla\mathcal{L}$: gradient of the loss
g^{+}: geometry-dependent gradient transformation

Stokes et al. (2020), Quantum Natural Gradient.

The broader quantum computing landscape

- **Managing quantum errors.** Quantum hardware is highly susceptible to environmental noise and imperfections in control and readout, with limited coherence times. Error suppression, mitigation and correction provide different approaches to managing quantum errors.

IBM: error mitigation and suppression

The broader quantum computing landscape

- **Managing quantum errors.** Quantum hardware is highly susceptible to environmental noise and imperfections in control and readout, with limited coherence times. Error suppression, mitigation and correction provide different approaches to managing quantum errors.
- **From circuits to hardware.** Execution of an abstract quantum circuit on hardware requires design choices about qubit placement, interactions between qubits, and decomposition into hardware-compatible gates. This process, called transpilation, adapts and optimizes the circuit for the chosen device.

IBM: error mitigation and suppression IBM: transpilation

The broader quantum computing landscape

- **Managing quantum errors.** Quantum hardware is highly susceptible to environmental noise and imperfections in control and readout, with limited coherence times. Error suppression, mitigation and correction provide different approaches to managing quantum errors.
- **From circuits to hardware.** Execution of an abstract quantum circuit on hardware requires design choices about qubit placement, interactions between qubits, and decomposition into hardware-compatible gates. This process, called transpilation, adapts and optimizes the circuit for the chosen device.
- **Beyond classification.** Quantum algorithms address tasks beyond classification, including encoding linear-system solutions in quantum states (HHL), search (Grover), quantum simulation, evaluating risks and expected outcomes (QAE), and optimization (QAOA).

IBM: error mitigation and suppression IBM: transpilation

The broader quantum computing landscape

- **Managing quantum errors.** Quantum hardware is highly susceptible to environmental noise and imperfections in control and readout, with limited coherence times. Error suppression, mitigation and correction provide different approaches to managing quantum errors.
- **From circuits to hardware.** Execution of an abstract quantum circuit on hardware requires design choices about qubit placement, interactions between qubits, and decomposition into hardware-compatible gates. This process, called transpilation, adapts and optimizes the circuit for the chosen device.
- **Beyond classification.** Quantum algorithms address tasks beyond classification, including encoding linear-system solutions in quantum states (HHL), search (Grover), quantum simulation, evaluating risks and expected outcomes (QAE), and optimization (QAOA).
- **Different computational models.** In common quantum-community nomenclature, "quantum computing" often refers to the gate-based model used in this talk, where computation proceeds through a sequence of discrete gates. Other quantum hardware platforms use different computational models, such as quantum annealing, which evolves a system under a continuously changing Hamiltonian to search for low-energy solutions. Familiarity with these models helps match problems to suitable hardware.

IBM: error mitigation and suppression IBM: transpilation

06 / 06

Guiding questions and concluding remarks

Questions to explore

- **Scaling up.** What design choices would you make when moving beyond three images and two sampled pixels per image? Consider feature reduction, encoding, circuit structure and classical components in a hybrid model.

Questions to explore

- **Scaling up.** What design choices would you make when moving beyond three images and two sampled pixels per image? Consider feature reduction, encoding, circuit structure and classical components in a hybrid model.
- **Fair comparisons.** How would you establish a fair comparison between quantum and classical models with different capacities, parameter counts and computational resources? How should the choice of classical baselines and evaluation criteria reflect the study's objectives and stage of development?

Questions to explore

- **Scaling up.** What design choices would you make when moving beyond three images and two sampled pixels per image? Consider feature reduction, encoding, circuit structure and classical components in a hybrid model.
- **Fair comparisons.** How would you establish a fair comparison between quantum and classical models with different capacities, parameter counts and computational resources? How should the choice of classical baselines and evaluation criteria reflect the study's objectives and stage of development?
- **Barren plateaus.** As the model grows, how would you test for barren plateaus and assess strategies for avoiding them?

Questions to explore

- **Scaling up.** What design choices would you make when moving beyond three images and two sampled pixels per image? Consider feature reduction, encoding, circuit structure and classical components in a hybrid model.
- **Fair comparisons.** How would you establish a fair comparison between quantum and classical models with different capacities, parameter counts and computational resources? How should the choice of classical baselines and evaluation criteria reflect the study's objectives and stage of development?
- **Barren plateaus.** As the model grows, how would you test for barren plateaus and assess strategies for avoiding them?
- **Hardware performance.** How do noise, finite-shot sampling and transpilation choices affect the model? How would you evaluate and improve these choices?

Questions to explore

- **Scaling up.** What design choices would you make when moving beyond three images and two sampled pixels per image? Consider feature reduction, encoding, circuit structure and classical components in a hybrid model.
- **Fair comparisons.** How would you establish a fair comparison between quantum and classical models with different capacities, parameter counts and computational resources? How should the choice of classical baselines and evaluation criteria reflect the study's objectives and stage of development?
- **Barren plateaus.** As the model grows, how would you test for barren plateaus and assess strategies for avoiding them?
- **Hardware performance.** How do noise, finite-shot sampling and transpilation choices affect the model? How would you evaluate and improve these choices?
- **Resources and cost.** What would training and inference cost on available quantum hardware? How would those costs shape the scope and design of the study?

Platform / hardware	Pricing model	Cost example (USD)
IBM Quantum / IBM QPUs	Pay-as-you-go; from $96 / minute	10 seconds of QPU usage ≈ $16
Amazon Braket / Rigetti Cepheus	$0.30 / task + $0.000425 / shot	1 task, 1,000 shots ≈ $0.73
Azure Quantum / IonQ Forte 1	Gate-shots; program minimum	Minimum $25.79; mitigation off

Prices as of 2 Oct 2026.

IBM pricing Amazon Braket pricing Microsoft Azure Quantum pricing

Looking back, looking ahead

- Covered the basics of machine learning and quantum computing, including encoding classical data as quantum states.

Looking back, looking ahead

- Covered the basics of machine learning and quantum computing, including encoding classical data as quantum states.
- Computed quantum similarity scores, built an SVM classifier, and trained variational and hybrid quantum–classical models to compare their predictions.

Looking back, looking ahead

- Covered the basics of machine learning and quantum computing, including encoding classical data as quantum states.
- Computed quantum similarity scores, built an SVM classifier, and trained variational and hybrid quantum–classical models to compare their predictions.
- Explored broader perspectives in QML, including barren plateaus, quantum natural gradient, other quantum algorithms and hardware considerations.

Looking back, looking ahead

- Covered the basics of machine learning and quantum computing, including encoding classical data as quantum states.
- Computed quantum similarity scores, built an SVM classifier, and trained variational and hybrid quantum–classical models to compare their predictions.
- Explored broader perspectives in QML, including barren plateaus, quantum natural gradient, other quantum algorithms and hardware considerations.
- For further exploration, try QML with datasets such as MNIST, Fashion-MNIST and EuroSAT, experimenting with different encodings, circuit architectures and hybrid models.

Looking back, looking ahead

- Covered the basics of machine learning and quantum computing, including encoding classical data as quantum states.
- Computed quantum similarity scores, built an SVM classifier, and trained variational and hybrid quantum–classical models to compare their predictions.
- Explored broader perspectives in QML, including barren plateaus, quantum natural gradient, other quantum algorithms and hardware considerations.
- For further exploration, try QML with datasets such as MNIST, Fashion-MNIST and EuroSAT, experimenting with different encodings, circuit architectures and hybrid models.
- To continue learning and stay connected with the field, explore educational resources such as the IBM Quantum machine-learning course, and take part in talks, workshops and hackathons.

Thanks!

Stay curious. Keep exploring.

Stay in touch!



Pale Blue Dot, NASA/JPL-Caltech

References

Books

M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information, 10th anniversary edition, 2010.

M. Schuld and F. Petruccione, Machine Learning with Quantum Computers, 2021.

R. Wolf, Quantum Key Distribution: An Introduction with Exercises, 2021.

Courses and educational resources

IBM, Quantum Machine Learning.

IBM, IBM Quantum Learning.

PennyLane, Learn quantum programming.

References

Documentation

IBM, Error mitigation and suppression techniques.

IBM, Introduction to transpilation.

scikit-learn developers, Support vector machines: mathematical formulation.

Hardware access and pricing

IBM, Quantum products and services.

Amazon Web Services, Amazon Braket pricing.

Microsoft, Azure Quantum pricing.

References

Y. LeCun, C. Cortes and C. J. C. Burges, MNIST database of handwritten digits, 1998.

H. Xiao, K. Rasul and R. Vollgraf, Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017.

O. M. Parkhi et al., Cats and Dogs (Oxford-IIIT Pet Dataset), 2012.

A. M. Horst, A. P. Hill and K. B. Gorman, palmerpenguins: Palmer Archipelago (Antarctica) penguin data, 2020.

P. Helber et al., EuroSAT: a novel dataset and deep learning benchmark for land use and land cover classification, 2019.

PennyLane, Quantum Data.

NASA/JPL-Caltech, Pale Blue Dot Revisited, 2020.

References

Trainability, geometry and capacity

M. Cerezo et al., Does provable absence of barren plateaus imply classical simulability?, 2025.

A. Abbas et al., The power of quantum neural networks, 2021.

J. Stokes et al., Quantum Natural Gradient, 2020.

J. R. McClean et al., Barren plateaus in quantum neural network training landscapes, 2018.

Variational algorithms and quantum optimization

A. Peruzzo et al., A variational eigenvalue solver on a quantum processor, 2014.

E. Farhi, J. Goldstone and S. Gutmann, A Quantum Approximate Optimization Algorithm, 2014.

T. Kadowaki and H. Nishimori, Quantum annealing in the transverse Ising model, 1998.